

Console-Based To-Do List Application

Interactive Task Management Tool built in Python

Intern	Nimra	Organization	Decode Labs
Program	Python Development Internship	Language	Python 3.12.4
Document Type	Project Documentation Report	Status	Completed

1. Project Title & Overview

The Console-Based To-Do List Application is an interactive Python program that lets a user build and review a personal task list directly from the terminal. The application starts with a pre-populated set of default tasks and allows the user to append new tasks through a simple, repeatable prompt loop. Once the user finishes adding entries, the complete, numbered task list is displayed as a clean summary.

2. Objectives & Features

Objectives

- Provide a straightforward CLI tool for capturing and reviewing personal or academic tasks.
- Demonstrate use of Python lists, loops, and conditional branching in a practical workflow.
- Allow flexible, repeated task entry through a user-controlled loop rather than a fixed number of prompts.
- Display a clearly numbered final task list for easy review.

Key Features

- Pre-loaded with a default set of sample tasks stored in a Python list.
- Interactive **y / n** prompt loop allowing the user to add any number of new tasks.
- Input normalization via **.strip().lower()** for consistent, case-insensitive responses.
- Input validation with a friendly error message for any response other than **y** or **n**.
- Automatic list numbering on output using **enumerate()**, starting from 1.
- Simple, dependency-free design using only core Python data structures.

3. System Requirements & Technologies Used

Item	Details
Language	Python 3.12.4
Standard Modules Used	None required — uses only Python built-ins (list , input() , enumerate() , print())
External Dependencies	None (uses only the Python Standard Library / built-ins)

Operating System	Windows / macOS / Linux (cross-platform)
Execution Environment	Any terminal / command prompt with Python 3 installed
Entry File	main.py

4. Code Architecture & Workflow

The program initializes a **tasks** list containing five default sample entries and a boolean flag **running** set to **True**, which controls the main program loop.

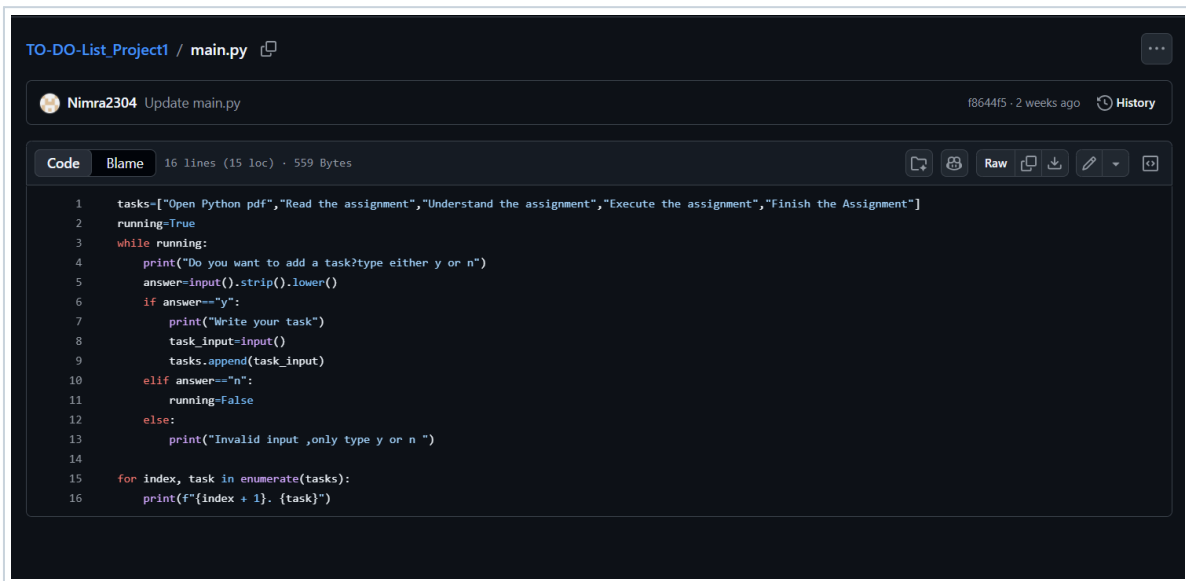
Inside the **while running:** loop, the user is asked whether they want to add a task. The response is normalized with **.strip().lower()** before being checked. If the answer is **"y"**, the user is prompted to type a new task, which is appended to the **tasks** list using **.append()** and the loop continues. If the answer is **"n"**, **running** is set to **False**, ending the loop. Any other input is rejected with a validation message, and the user is prompted again on the next iteration.

Once the loop exits, the program iterates over the final **tasks** list using **enumerate(tasks)** and prints each task with a human-friendly, 1-based index (via **index + 1**), producing a clean, numbered summary of every task entered during the session.

5. How to Run the Program (CLI)

- 1 Ensure Python 3.12.4 (or a compatible Python 3 release) is installed by running **python --version** (or **python3 --version**) in your terminal.
- 2 Clone or download the repository: **git clone https://github.com/Nimra2304/TO-DO-List_Project1.git**
- 3 Navigate into the project directory: **cd TO-DO-List_Project1**
- 4 Run the script: **python main.py** (use **python3 main.py** on macOS/Linux).
- 5 When prompted **Do you want to add a task? type either y or n**, type **y** to add a task or **n** to finish.
- 6 If you typed **y**, enter the task text when prompted with **Write your task**, then repeat as needed.
- 7 Once you type **n**, the complete numbered task list is printed to the console.

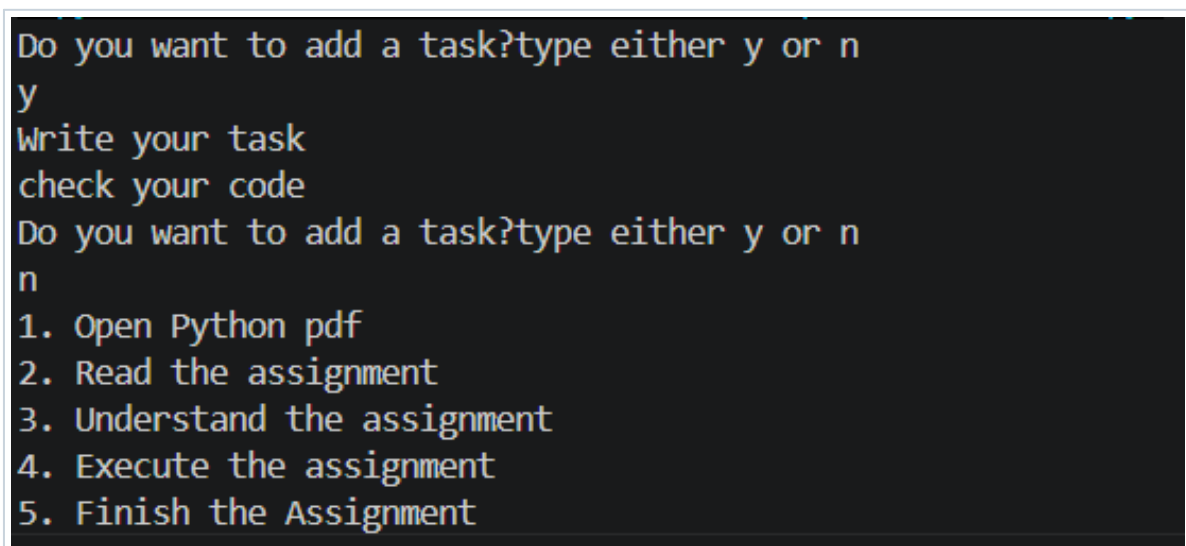
6. Screenshots



The screenshot shows the GitHub repository view for the file `main.py` in the `TO-DO-List_Project1` repository. The file is 16 lines long, 15 lines of code, and 559 bytes. The code is as follows:

```
1 tasks=["Open Python pdf","Read the assignment","Understand the assignment","Execute the assignment","Finish the Assignment"]
2 running=True
3 while running:
4     print("Do you want to add a task?type either y or n")
5     answer=input().strip().lower()
6     if answer=="y":
7         print("Write your task")
8         task_input=input()
9         tasks.append(task_input)
10    elif answer=="n":
11        running=False
12    else:
13        print("Invalid input ,only type y or n ")
14
15 for index, task in enumerate(tasks):
16     print(f"{index + 1}. {task}")
```

Figure 1 — `main.py` source code (GitHub repository view).



The screenshot shows a terminal window with the following output:

```
Do you want to add a task?type either y or n
y
Write your task
check your code
Do you want to add a task?type either y or n
n
1. Open Python pdf
2. Read the assignment
3. Understand the assignment
4. Execute the assignment
5. Finish the Assignment
```

Figure 2 — Sample terminal execution: adding a task and viewing the numbered list.

```
Write your task
check your code
Do you want to add a task?type either y or n
n
1. Open Python pdf
2. Read the assignment
3. Understand the assignment
4. Execute the assignment
5. Finish the Assignment
6. check your code
```

Figure 3 — Continued execution: additional task added, showing the updated final list.

7. GitHub Repository Link

https://github.com/Nimra2304/TO-DO-List_Project1/tree/main